

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA0706

COURSE OUTCOME COVERED

CO4: Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards (**BL2, BL6**)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

Coding challenge

Code of Conduct:

I **M Nishanth (192511154)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

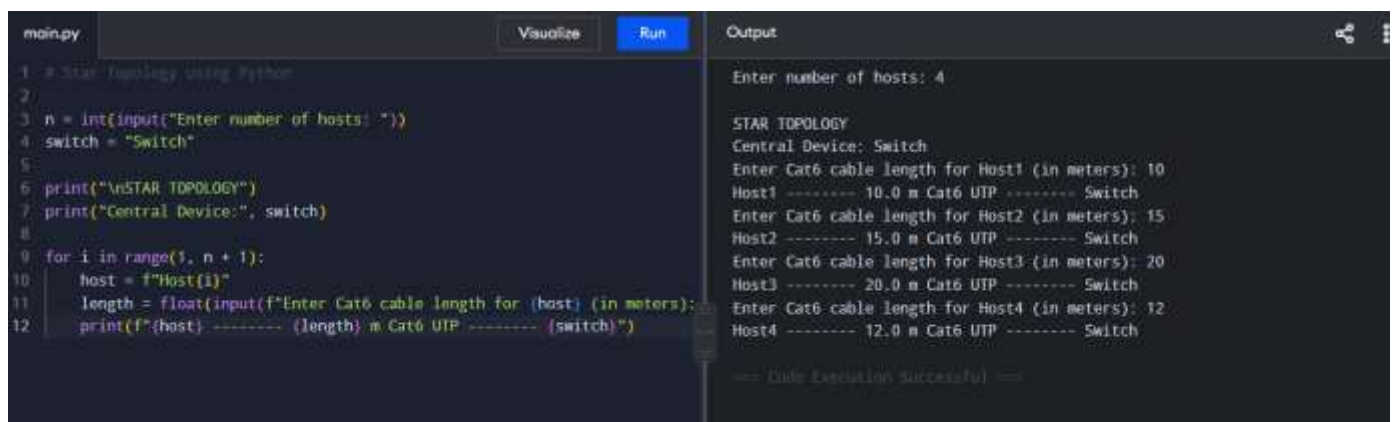
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Nishanth

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a star topology by taking the number of hosts and a central switch as input, then printing the details for each host and the switch. The output window shows the execution results for 4 hosts.

```
1 # Star topology using Python
2
3 n = int(input("Enter number of hosts: "))
4 switch = "Switch"
5
6 print("\nSTAR TOPOLOGY")
7 print("Central Device:", switch)
8
9 for i in range(1, n + 1):
10     host = f"Host{i}"
11     length = float(input(f"Enter Cat6 cable length for {host} (in meters): "))
12     print(f"{host} ----- {length} m Cat6 UTP ----- {switch}")
```

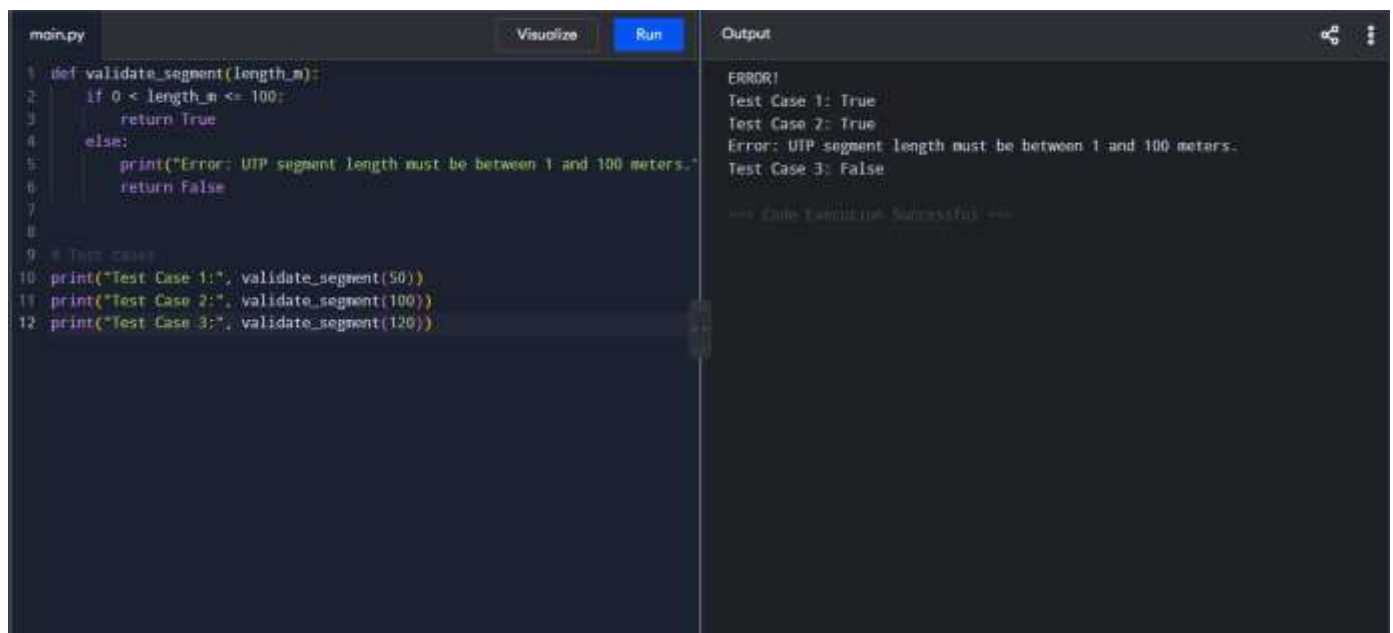
Output:

```
Enter number of hosts: 4

STAR TOPOLOGY
Central Device: Switch
Enter Cat6 cable length for Host1 (in meters): 10
Host1 ----- 10.0 m Cat6 UTP ----- Switch
Enter Cat6 cable length for Host2 (in meters): 15
Host2 ----- 15.0 m Cat6 UTP ----- Switch
Enter Cat6 cable length for Host3 (in meters): 20
Host3 ----- 20.0 m Cat6 UTP ----- Switch
Enter Cat6 cable length for Host4 (in meters): 12
Host4 ----- 12.0 m Cat6 UTP ----- Switch

== Code Execution Successful ==
```

2. Code a function `validate\_segment(length\_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function to validate UTP segment lengths and includes three test cases. The output window shows the results of these tests.

```
1 def validate_segment(length_m):
2     if 0 < length_m <= 100:
3         return True
4     else:
5         print("Error: UTP segment length must be between 1 and 100 meters.")
6         return False
7
8
9 # Test cases
10 print("Test Case 1:", validate_segment(50))
11 print("Test Case 2:", validate_segment(100))
12 print("Test Case 3:", validate_segment(120))
```

Output:

```
ERROR!
Test Case 1: True
Test Case 2: True
Error: UTP segment length must be between 1 and 100 meters.
Test Case 3: False

== Code Execution Successful ==
```

3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma\_log.txt`.

csma.py - C:/Users/M NISHANTH/Downloads/csma.py (3.12.7)

File Edit Format Run Options Window Help

```
import random
import time

hosts = ["Host1", "Host2", "Host3", "Host4"]

with open("csma_log.txt", "w") as log:
    log.write("CSMA/CD Simulation - Star Topology\n")
    log.write("-----\n")

    for i in range(8):
        host = random.choice(hosts)

        # Randomly decide whether collision occurs
        collision = random.choice([True, False])

        if collision:
            backoff = random.randint(1, 10)

            message = (
                f"Collision detected by {host} | "
                f"Back-off interval = {backoff} ms"
            )

            print(message)
            log.write(message + "\n")

            time.sleep(backoff / 1000)

        else:
            message = f"{host} transmitted successfully | No collision"
            print(message)
            log.write(message + "\n")

print("\nSimulation completed.")
print("Log saved in csma_log.txt")
```

```

IDLE Shell 3.12.7
File Edit Shell Debug Options Window Help
Python 3.12.7 (tags/v3.12.7:0b05ead, Oct 1 2024, 03:06:41) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/M NISHANTH/Downloads/csma.py =====
Collision detected by Host2 | Back-off interval = 5 ms
Collision detected by Host4 | Back-off interval = 4 ms
Host4 transmitted successfully | No collision
Host2 transmitted successfully | No collision
Collision detected by Host2 | Back-off interval = 10 ms
Host2 transmitted successfully | No collision
Host1 transmitted successfully | No collision
Host4 transmitted successfully | No collision

Simulation completed.
Log saved in csma_log.txt
>>>

```

4. Using Python's 'socket' library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.

```

# star.py - C:/Users/M NISHANTH/OneDrive/Desktop/EXPERIMENT 4/server.py (C:\Python312)
File Edit Shell Debug Options Window Help
import socket

HOST = "127.0.0.1"
PORT = 5500

# Simulated switch MAC address table
mac_table = {
    "AA-AA-AA-AA:01": "Host1",
    "AA-AA-AA-AA:02": "Host2"
}

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen()

print("Star Switch is ON")
print("Switch MAC Address Table:")
print(mac_table)

conn, addr = server.accept()
print(f"\nHost1 connected to the switch.")

while True:
    data = conn.recv(1024)

    if not data:
        break

    message = data.decode()

    print(f"\nFrame received by Switch:")
    print(f"Source MAC : AA-AA-AA-AA:01")
    print(f"Destination MAC : AA-AA-AA-AA:02")
    print(f"Message : {message}")

    print(f"\nSwitch forwarding frame from Host1 to Host2...")

    conn.send(f"Message forwarded successfully to Host2".encode())

    print(f"\nSwitch MAC Address Table after frame transmission:")
    for mac, host in mac_table.items():
        print(f"mac: {mac} | host: {host}")

conn.close()
server.close()

```

```

IDLE Shell 3.12.7
File Edit Shell Debug Options Window Help
Python 3.12.7 (tags/v3.12.7:0b05ead, Oct 1 2024, 03:06:41) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/M NISHANTH/OneDrive/Desktop/EXPERIMENT 4/server.py =====
Star Switch is ON
Switch MAC Address Table:
{'AA-AA-AA-AA:01': 'Host1', 'AA-AA-AA-AA:02': 'Host2'}

```

```
client.py - C:/Users/M NISHANTH/OneDrive/Desktop/client.py (3,12,7)
File Edit Format Run Options Window Help
import socket

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

print("Host1 connected to the Star Switch.")

for i in range(3):
    message = input("Enter message to Host2: ")
    client.send(message.encode())
    response = client.recv(1024).decode()
    print("Switch:", response)

client.close()
print("Connection closed.")

Ln 21 Col 0
```

```
IDLE Shell 3.12.7
File Edit Shell Debug Options Window Help
Python 3.12.7 (tags/v3.12.7:0b05ead, Oct 1 2024, 03:06:41) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/M NISHANTH/OneDrive/Desktop/client.py =====
Host1 connected to the Star Switch.
Enter message to Host2: hii
Switch: Message forwarded successfully to Host2
Enter message to Host2: hello
Switch: Message forwarded successfully to Host2
Enter message to Host2: how are you
Switch: Message forwarded successfully to Host2
Connection closed.
>>>

Ln 13 Col 0
```

```
client
IDLE Shell 3.12.7
File Edit Shell Debug Options Window Help
Python 3.12.7 (tags/v3.12.7:0b05ead, Oct 1 2024, 03:06:41) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/M NISHANTH/OneDrive/Desktop/client.py =====
Host1 connected to the Star Switch.
Enter message to Host2: hii
Switch: Message forwarded successfully to Host2
Enter message to Host2: hello
Switch: Message forwarded successfully to Host2
Enter message to Host2: how are you
Switch: Message forwarded successfully to Host2
Connection closed.
>>>

Ln 13 Col 0
```

```
IDLE Shell 3.12.7
File Edit Shell Debug Options Window Help
===== RESTART: C:/Users/M NISHANTH/OneDrive/Desktop/EXPERIMENT 4/server.py =====
Star Switch is ON
Switch MAC Address Table:
{'AA:AA:AA:AA:AA:01': 'Host1', 'AA:AA:AA:AA:AA:02': 'Host2'}

Host2 connected to the switch.

Frame received by Switch:
Source MAC : AA:AA:AA:AA:AA:01
Destination MAC : AA:AA:AA:AA:AA:02
Message : hii

Switch forwarding frame from Host1 to Host2...

Switch MAC Address Table after frame transmission:
AA:AA:AA:AA:AA:01 -> Host1
AA:AA:AA:AA:AA:02 -> Host2

Frame received by Switch:
Source MAC : AA:AA:AA:AA:AA:01
Destination MAC : AA:AA:AA:AA:AA:02
Message : hello

Switch forwarding frame from Host1 to Host2...

Switch MAC Address Table after frame transmission:
AA:AA:AA:AA:AA:01 -> Host1
AA:AA:AA:AA:AA:02 -> Host2

Frame received by Switch:
Source MAC : AA:AA:AA:AA:AA:01
Destination MAC : AA:AA:AA:AA:AA:02
Message : how are you

Switch forwarding frame from Host1 to Host2...

Switch MAC Address Table after frame transmission:
AA:AA:AA:AA:AA:01 -> Host1
AA:AA:AA:AA:AA:02 -> Host2
>>>

Ln 43
```

5. Create a class `StarSwitch` with methods `add\_port(host\_id)`, `remove\_port(host\_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

```
exp5.py - C:/Users/M NISHANTH/OneDrive/Desktop/EXPERIMENT 4/exp5.py (3.12.7)
File Edit Format Run Options Window Help

    print(f"{host_id} connected to the switch.")

def remove_port(self, host_id):
    if host_id in self.ports:
        self.ports.remove(host_id)
        print(f"{host_id} removed from the switch.")
    else:
        print(f"{host_id} is not connected.")

def broadcast(self, frame):
    source = frame["source"]
    message = frame["message"]

    print(f"\nBroadcasting frame from {source}: {message}")

    for host in self.ports:
        if host != source:
            print(f"Frame delivered to {host}")

# Create a star switch
switch = StarSwitch()

# Add hosts
switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")

# Broadcast a frame
frame = {
    "source": "Host1",
    "message": "Hello to all hosts"
}

switch.broadcast(frame)

# Remove a host
switch.remove_port("Host4")

# Broadcast another frame
frame2 = {
    "source": "Host2",
    "message": "Network message"
}

switch.broadcast(frame2)
```

IDLE Shell 3.12.7

File Edit Shell Debug Options Window Help

python 3.12.7 (tags/v3.12.7:10005ead, OCT 1 2024, 03:06:41) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>

===== RESTART: C:/Users/M KISHAN/OneDrive/Desktop/EXPERIMENT 4/exp5.py =====

Host1 connected to the switch.
Host2 connected to the switch.
Host3 connected to the switch.
Host4 connected to the switch.

Broadcasting frame from Host1: Hello to all hosts
Frame delivered to Host2
Frame delivered to Host3
Frame delivered to Host4
Host4 removed from the switch.

Broadcasting frame from Host2: Network message
Frame delivered to Host1
Frame delivered to Host3

>>>

1. Create a class `StarSwitch` with methods `add_port(host_id)`, `remove_port(host_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done